# Generic-DLP correctness fixes: collab 2-cycle escape, Pohlig–Hellman

Study `research/generic_dlp_correctness_20261017` · 2026-10-17 · counts only, nothing timed

**What changed.** Findings 3–6 of `research/generic_dlp_memory_parallelism_20261017/AUDIT_FINDINGS.md`.

1. **`pollard_collab` negation-map 2-cycle escape.**
   - The 2-cycle test compared the unflipped sum with the normalised previous point, so it never fired.
   - It now compares after the flip and escapes by doubling the smaller-x member of the cycle.
2. **Pohlig–Hellman digits by baby-step giant-step.**
   - Each digit was a linear search over `[0, q)`.
   - Now one `⌈√q⌉`-point table per prime serves all of its digits.
   - Each digit is still the smallest solution below `q`, so the residues are unchanged.
3. **Pohlig–Hellman factorisation bounded.** Trial division now stops at the smoothness bound.
4. **Pohlig–Hellman answers checked.** `[d]G = Q` is verified, and order 1 yields `d = 0`.

**Arms.**
- Before: `main` at `b1bf5b8935f68257dfdd3fc542cb2fef41db741c`.
- After: `claude/generic-dlp-correctness-fixes`.
- Both arms run the same driver, `examples/generic_dlp_correctness_counts.rs`.
- Build: cargo build --release (default profile) rustc 1.97.0 (2d8144b78 2026-07-07).

## 1. Collab walkers

| curve | negation map | walkers | DPs before → after | dead trails before → after | all DPs verify | unchanged |
|---|---|---|---|---|---|---|
| demo-small | no | 512 | 512 → 512 | 0 → 0 | yes | yes |
| demo-small | yes | 512 | 462 → 511 | 50 → 1 | yes | no |
| demo-mid | no | 512 | 511 → 511 | 1 → 1 | yes | yes |
| demo-mid | yes | 512 | 406 → 512 | 106 → 0 | yes | no |
| demo-32 | no | 256 | 256 → 256 | 0 → 0 | yes | yes |
| demo-32 | yes | 256 | 59 → 241 | 197 → 15 | yes | no |

- **Without the negation map the code path is not taken**, and every count is identical.
- **With it**, walkers that used to die at the step cap in fruitless 2-cycles now leave the cycle and reach distinguished points.
- **What still dies** was not classified walker by walker. Longer fruitless cycles, which this fix does not handle, are the expected cause, and the `20·2^dp_bits` step cap still bounds them.

## 2. Pohlig–Hellman

**The panel.**
- Curve: $y^2 = x^3 + 4$ over a 47-bit prime, of order $3 \cdot 19^4 \cdot 151 \cdot 181 \cdot 13171$.
- 24 targets × 2 bounds, 48 solves in all.
- All correct before and after: yes.
- Recovered logarithms identical: yes.

**Result.** `total_steps` went from 284364 to 10682 (26.62×). Per solve the ratio ranges 1.85–46.53×.

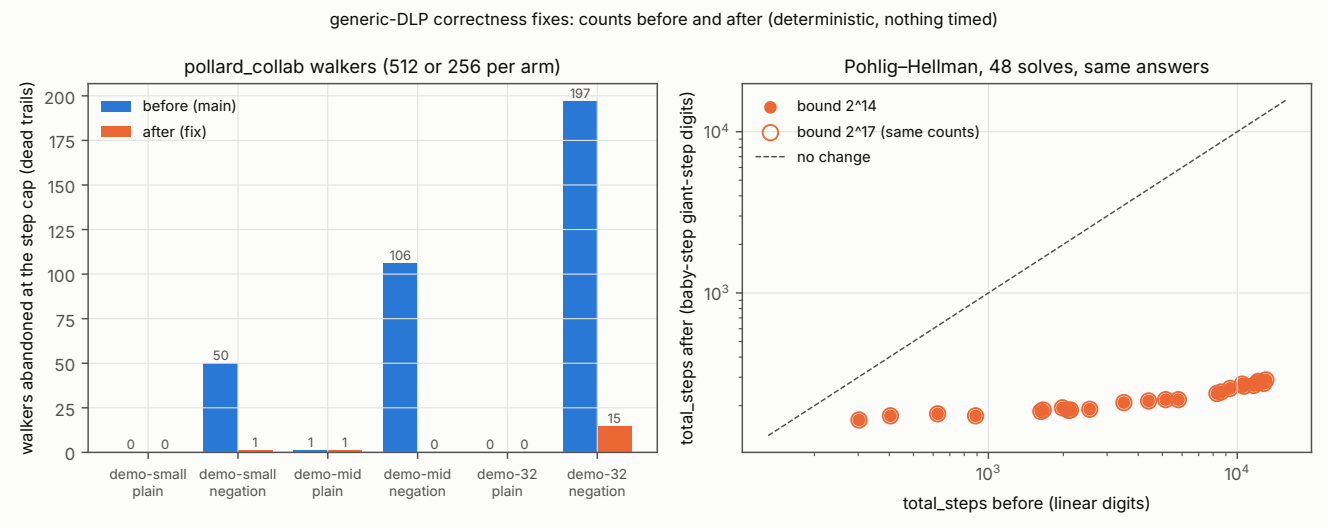



Figure 1: Left: dead trails per curve and mode, before and after. Right: Pohlig–Hellman `total_steps` per solve, before against after (log–log); the dashed line is no change.

## 3. Pinned walk test re-pin (`tests/ec_vartime_walks_pinned.rs`)

The transcripts the test hashes were captured before and after, in `transcripts/`. The "before" run used `main`'s sources with the digests zeroed, and it reproduced the pinned digests exactly.

**Collab.**

- `neg=false` sections byte-identical: yes.
- Dead trails in the `neg=true` sections: 30 ⟶ 16.

**Pohlig–Hellman and Cheon.**

- Identical apart from step counts: yes.
- The Cheon lines and every answer, factorisation and residue are unchanged, wrong-order cases included.
- Over 72 Pohlig–Hellman lines, `total_steps` went from 218517 to 8648.

| section | DP before | dead before | DP after | dead after | identical |
|---|---|---|---|---|---|
| demo-32 neg=false dp=10 | 6 | 0 | 6 | 0 | yes |
| demo-32 neg=false dp=8 | 6 | 0 | 6 | 0 | yes |
| demo-32 neg=true dp=10 | 1 | 5 | 3 | 3 | no |
| demo-32 neg=true dp=8 | 2 | 4 | 6 | 0 | no |
| demo-mid neg=false dp=4 | 24 | 0 | 24 | 0 | yes |
| demo-mid neg=false dp=6 | 24 | 0 | 24 | 0 | yes |
| demo-mid neg=true dp=4 | 20 | 4 | 21 | 3 | no |
| demo-mid neg=true dp=6 | 16 | 8 | 21 | 3 | no |
| demo-small neg=false dp=3 | 23 | 1 | 23 | 1 | yes |
| demo-small neg=false dp=5 | 24 | 0 | 24 | 0 | yes |
| demo-small neg=true dp=3 | 23 | 1 | 24 | 0 | no |
| demo-small neg=true dp=5 | 16 | 8 | 17 | 7 | no |

## 4. Scope

- **Class.** The collab fix is a correctness fix that changes walk transcripts by design. The Pohlig–Hellman change is an algorithmic change to its counted cost (`O(e·q)` ⟶ `O((e+1)√q)` per prime). Neither touches `S` or any IC/rho comparison.
- **What the counts cover.** They cover these curves and targets only. How often fruitless 2-cycles arise elsewhere depends on the branch count and `dp_bits`.
- **What is not measured.** Pohlig–Hellman wall time; the counts are the metric.